

HPC Notes- Unit 1 & 2

UNIT 1: INTRODUCTION TO PARALLEL COMPUTING

BLOCK 1: WHY PARALLEL COMPUTING?

The Problem with Traditional Computing

- **1986-2002:** Microprocessors improved by **50% per year**
- **After 2002:** Dropped to only **~20% per year**
- **Reason:** Physics limitations
 - Smaller transistors → Faster processors
 - Faster processors → More power consumption
 - More power → More heat
 - More heat → Unreliable processors

The Solution

- Instead of faster single processors → **Multiple processors on one chip** (Multicore)
- This introduced **PARALLELISM**

Why We Need More Performance

- Climate modeling
 - Protein folding
 - Drug discovery
 - Energy research
 - Data analysis
-

BLOCK 2: PARALLEL PROGRAMMING BASICS

Why We Must Write Parallel Programs

- Serial programs **don't benefit** from multiple cores
- Running multiple instances of same program isn't useful
- Automatic conversion from serial to parallel is **very difficult**

Two Main Approaches to Parallelism

Type	Description	Example
Task Parallelism	Partition different tasks among cores	Grading: one core checks Q1, another checks Q2
Data Parallelism	Partition data among cores, same operation	Each core processes part of an array

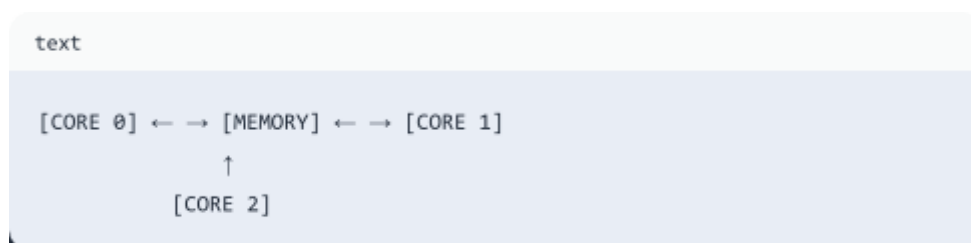
Coordination Requirements

1. **Communication:** Cores share/send data
2. **Load Balancing:** Equal work distribution
3. **Synchronization:** Cores don't get too far ahead/behind

BLOCK 3: PARALLEL SYSTEM TYPES

Shared Memory Systems

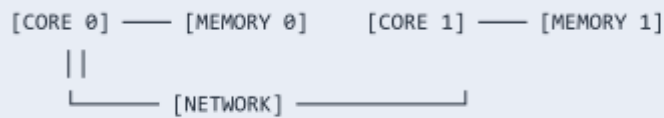
- All cores access the **same memory**
- Communicate by reading/writing shared variables
- **Example:** Multicore processors



Distributed Memory Systems

- Each core has **private memory**
- Communicate by **sending messages** across network
- **Example:** Computer clusters

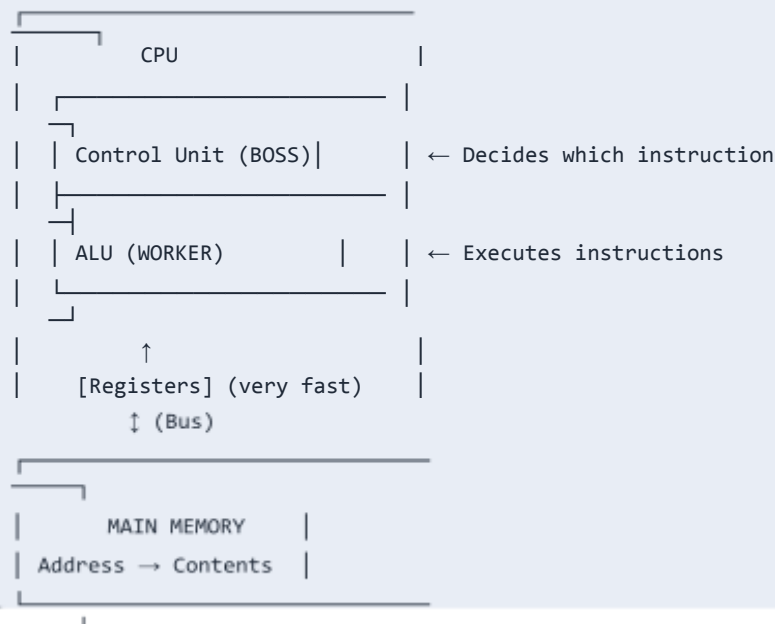
text



BLOCK 4: THE VON NEUMANN ARCHITECTURE

Key Components

text



Important Terms

- **Register:** Ultra-fast storage inside CPU
- **Program Counter:** Stores address of next instruction
- **Bus:** Connects CPU and memory
- **von Neumann Bottleneck:** The bus is the slow point!

BLOCK 5: PROCESSES AND THREADS

Process

- An instance of a running program

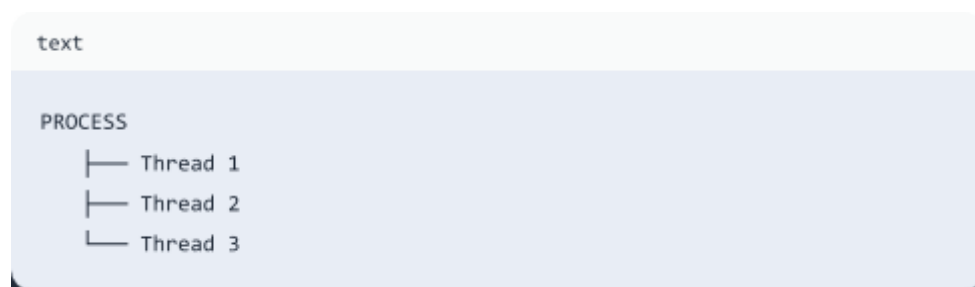
- Contains: executable code, memory block, resources, security info, state

Multitasking

- Single processor runs multiple programs by **time-slicing**
- Each process gets a turn, then blocks/wait

Threading

- **Threads** are contained within **processes**
- Allow dividing programs into independent tasks
- When one thread blocks, another can run



BLOCK 6: CACHING

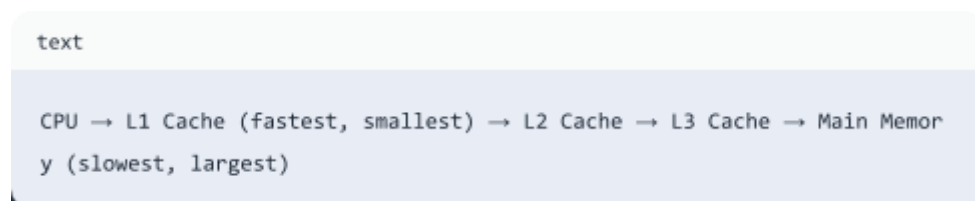
What is Cache?

- Memory that can be accessed **faster** than main memory
- Located **on the CPU chip**

Principle of Locality

- **Spatial Locality**: Access nearby locations
- **Temporal Locality**: Access same location again soon

Cache Levels



Cache Write Policies

- **Write-through**: Update memory immediately

- **Write-back:** Mark as "dirty", write later when replaced

Cache Mappings

Type	Description
Fully Associative	Any line can go anywhere
Direct Mapped	Each line has ONE location
n-way Set Associative	Each line can go in n locations

BLOCK 7: VIRTUAL MEMORY

Purpose

- Programs may be larger than physical memory
- Acts as a **cache for secondary storage**

How It Works

1. Program uses **virtual page numbers**
2. Page table maps virtual → physical addresses
3. Only **active parts** stay in main memory
4. Idle parts go to **swap space** on disk

Page Table



TLB (Translation Lookaside Buffer)

- Caches page table entries
- Speeds up address translation
- **Page fault:** Page is on disk, not in memory

BLOCK 8: INSTRUCTION LEVEL PARALLELISM (ILP)

Pipelining

- Functional units arranged in **stages**
- Each stage works on different instruction
- **Example:** Floating point addition (7 stages)

text

Without Pipelining: 7ns per addition → 7000ns for 1000 additions With
Pipelining: 7ns per addition → 1006ns for 1000 additions

Multiple Issue

- Replicate functional units
- Execute multiple instructions **simultaneously**
- **Static:** Scheduled at compile time
- **Dynamic:** Scheduled at run-time

Speculation

- Guess about branch outcome
- Execute based on guess
- If wrong → recalculate

BLOCK 9: FLYNN'S TAXONOMY

Type	Description	Example
SISD	Single Instruction, Single Data	Traditional CPU
SIMD	Single Instruction, Multiple Data	Vector processors, GPUs
MISD	Multiple Instruction, Single Data	Rare
MIMD	Multiple Instruction, Multiple Data	Most parallel systems

BLOCK 10: SIMD AND VECTOR PROCESSORS

SIMD

- Same instruction applied to **multiple data items**
- **Data parallelism**
- All ALUs execute same instruction or idle

Vector Processors

- Operate on **arrays/vectors** of data
- Features:
 - **Vector registers** (store vectors)
 - **Vector instructions** (operate on vectors)
 - **Interleaved memory** (fast access)
 - **Strided memory access**

GPU (Graphics Processing Unit)

- Uses SIMD parallelism
- Programmable **shader functions**
- Excellent for graphics and scientific computing

BLOCK 11: INTERCONNECTION NETWORKS

Shared Memory Interconnects

Type	Description	Pros/Cons
Bus	Shared communication wires	Simple, but contention increases with devices
Crossbar	Switches connect processors to memory	Fast, simultaneous communication, but expensive

Distributed Memory Interconnects

Type	Description
Direct	Switches connect directly to processors
Indirect	Switches not directly connected to processors

Network Metrics

Term	Definition
Bisection Width	Number of links connecting two halves
Bandwidth	Data rate of a link
Bisection Bandwidth	Sum of bandwidth across halves
Latency	Time to start receiving first byte
Message Time	= Latency + $n/\text{Bandwidth}$

BLOCK 12: CACHE COHERENCE

The Problem

- Multiple cores have their own caches
- Same variable may have **different values** in different caches

Solution 1: Snooping (Bus-based)

- All cores "snoop" the bus
- When a variable updates, broadcast the change
- Other cores mark their copy as **invalid**

Solution 2: Directory-based

- Maintain a **directory** of cache line status
- When updating, consult directory
- Invalidate all copies in other caches

Race Condition Example

text


```
Thread 0: x += my_val      (x = 0, my_val =  
7) Thread 1: x += my_val  (x = 0, my_val =  
19)
```

Possible Result: x = 7 OR x = 19 OR x = 26?!

BLOCK 13: PARALLEL SOFTWARE CONCEPTS

SPMD (Single Program Multiple Data)

- One executable
- Uses conditional branches to behave differently

Shared Memory Programs

- Start **single process**
- **Fork threads**
- Threads carry out tasks

Distributed Memory Programs

- Start **multiple processes**
- Each has private memory
- Communicate via messages

Thread Types

- **Dynamic:** Created when needed, terminate after work
- **Static:** Created once, reused (better performance)

Nondeterminism

- Race conditions occur when order of execution affects result
- **Critical Section:** Code that must execute atomically
- **Mutual Exclusion:** Only one thread at a time

BLOCK 14: PERFORMANCE MEASUREMENT

Speedup

text

$$\text{Speedup (S)} = T_{\text{serial}} / T_{\text{parallel}}$$

Efficiency

text

$$\text{Efficiency (E)} = S / p$$

where p = number of cores

Example Speedup Table

p	1	2	4	8
S	1.0	1.9	3.6	6.5
E	1.0	0.95	0.90	0.81

Amdahl's Law

- If fraction **f** of program is parallelizable:

text

$$T_{\text{parallel}} = f \times T_{\text{serial}} / p + (1-f) \times T_{\text{serial}}$$

$$\text{Speedup} = 1 / (f/p + 1-f)$$

- Limitation:** Even with infinite cores, speedup limited by serial portion!

Scalability

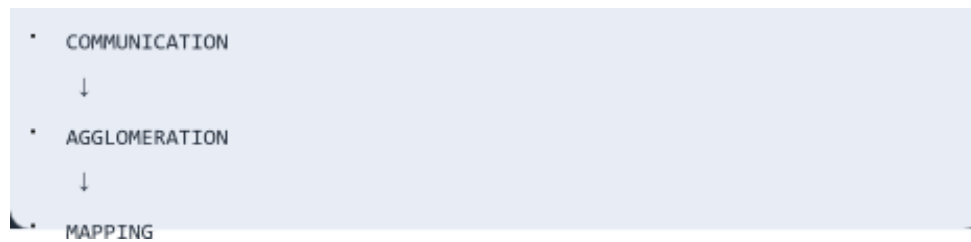
- Strong Scaling:** Speedup with fixed problem size
- Weak Scaling:** Speedup with proportional problem size

BLOCK 15: FOSTER'S PARALLEL PROGRAM DESIGN

4-Step Methodology

text

1. PARTITIONING
↓



Step	Description
1. Partitioning	Divide computation/data into small tasks
2. Communication	Determine what data must be shared
3. Agglomeration	Combine tasks (e.g., dependent tasks together)
4. Mapping	Assign tasks to processes/threads (minimize communication, balance work)

UNIT 2: SHARED MEMORY PROGRAMMING WITH OPENMP

BLOCK 1: OPENMP BASICS

What is OpenMP?

- **API** for shared-memory parallel programming
- **MP** = Multiprocessing
- Uses **pragmas** (special preprocessor instructions)
- Compilers that don't support OpenMP simply **ignore** pragmas

Key Features

- Fork-join model
- Starts multiple **threads**
- All threads share memory

Basic Program Structure

c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Hello(void);

int main(int argc, char* argv[]) {
    int thread_count = strtol(argv[1], NULL, 10);

    #pragma omp parallel num_threads(thread_count)
    Hello();

    return 0;
}

void Hello(void) {
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();
    printf("Hello from thread %d of %d\n", my_rank, thread_count);
}
```

Compilation

bash

```
gcc -g -Wall -fopenmp -o omp_hello omp_hello.c
./omp_hello 4
```

BLOCK 2: OPENMP TERMINOLOGY

Term	Meaning
Team	Collection of threads in parallel block
Master	Original thread (thread 0)
Parallel Block	Code executed by all threads
Pragma	#pragma omp ... directive
Clause	Text that modifies directive

The Parallel Directive

```
c

#pragma omp parallel num_threads(thread_count)
{
    // Structured block - all threads execute this
}
```

Thread Functions

```
c

int omp_get_thread_num();    // Get current thread ID
int omp_get_num_threads();  // Get total threads in team
```

Compiler Compatibility

```
c

#ifdef _OPENMP
    #include <omp.h>
#else
    // OpenMP not supported
#endif
```

BLOCK 3: THE TRAPEZOIDAL RULE

Serial Algorithm

```
c

h = (b-a)/n;
approx = (f(a) + f(b))/2.0; for
(i = 1; i <= n-1; i++) {
    x_i = a + i*h;
    approx += f(x_i);
}
approx = h*approx;
```

Parallel Design Steps

1. **Tasks:** Compute individual trapezoids + Sum them

2. **Communication:** Each trapezoid task sends result to sum task
3. **Agglomeration:** Assign contiguous blocks to each thread
4. **Mapping:** One thread per core

Parallel Implementation

```
c

void Trap(double a, double b, int n, double* global_result_p) { double
    h, x, my_result;
    double local_a, local_b;
    int i, local_n;
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();

    h = (b-a)/n;
    local_n = n/thread_count;
    local_a = a + my_rank*local_n*h;
    local_b = local_a + local_n*h;

    my_result = (f(local_a) + f(local_b))/2.0;
    for (i = 1; i <= local_n-1; i++) {
        x = local_a + i*h;
        my_result += f(x);
    }
    my_result = my_result*h;

    #pragma omp critical
    *global_result_p += my_result;
}
```

BLOCK 4: MUTUAL EXCLUSION

The Problem: Race Condition

```
text

Thread 0: global_result += my_result;    // my_result =
7 Thread 1: global_result += my_result;    // my_result =
19
```

Possible Results: 7, 19, or 26!

Solutions

Method	Syntax	When to Use
Critical	<code>#pragma omp critical</code>	General critical sections
Atomic	<code>#pragma omp atomic</code>	Simple operations like <code>x++</code>
Named Critical	<code>#pragma omp critical(name)</code>	Different critical sections
Locks	<code>omp_set_lock() / omp_unset_lock()</code>	Dynamic control

Critical Section Example

```
c
#pragma omp critical
global_result += my_result;
```

Atomic Example

```
c
#pragma omp atomic
x += 1;
```

Lock Example

```
c
omp_set_lock(&my_lock);
// Critical section
omp_unset_lock(&my_lock);
```

BLOCK 5: VARIABLE SCOPE

Scope Types

Scope	Description	Default?
Shared	Accessible by all threads	Variables declared before parallel block
Private	Only accessible by one thread	Loop index variables

Managing Scope

```
c
#pragma omp parallel default(none) private(i, factor) shared(n)
{
    // Must specify scope for ALL variables
}
```

The Reduction Clause

```
c
#pragma omp parallel for reduction(+: global_result)
for (i = 0; i < n; i++) {
    global_result += f(i);
}
```

Supported Operators:

+, *, -, /, %, ^, &, ||

&, |, ^, &&, ||

BLOCK 6: PARALLEL FOR LOOPS

Basic Syntax

```
c
#pragma omp parallel for num_threads(thread_count) \
    reduction(+: approx)
for (i = 1; i <= n-1; i++) {
    approx += f(a + i*h);
}
```


Legal For Loop Forms

c

```
// Valid forms
for (i = start; i < end; i++) { } for
(i = start; i <= end; i++) { } for (i
= start; i >= end; i--) { }
for (i = start; i > end; i--) { }

// Increment can be: i++, ++i, i--, --i, i += inc, i -= inc
```

Data Dependencies - DANGER!

```
c

// WRONG - Has data dependency!
fibo[0] = fibo[1] = 1;
#pragma omp parallel for
for (i = 2; i < n; i++) {
    fibo[i] = fibo[i-1] + fibo[i-2]; // Depends on previous results!
}
```

BLOCK 7: SCHEDULING LOOPS

Schedule Types

Type	Description	Best For
static	Iterations pre-assigned before execution	Uniform work, predictable
dynamic	Threads request chunks at runtime	Uneven work, unpredictable
guided	Decreasing chunk sizes	Less overhead than dynamic
auto	Compiler/runtime decides	When you don't know
runtime	Determined by environment variable	Testing different schedules

Static Schedule Examples (12 iterations, 3 threads)

text

```

schedule(static, 1):
Thread 0: 0, 3, 6, 9
Thread 1: 1, 4, 7, 10
Thread 2: 2, 5, 8, 11

schedule(static, 4):
Thread 0: 0, 1, 2, 3
Thread 1: 4, 5, 6, 7
Thread 2: 8, 9, 10, 11

```

Dynamic Schedule

```

c

#pragma omp parallel for schedule(dynamic, 1) for
(i = 0; i < n; i++) {
    sum += f(i); // Work varies per iteration
}

```

Environment Variable for Runtime

```

bash

export OMP_SCHEDULE="dynamic,4"

```

BLOCK 8: SORTING WITH OPENMP

Odd-Even Transposition Sort

```

c

for (phase = 0; phase < n; phase++) { if
    (phase % 2 == 0) {
        // Even phase: compare even-odd pairs
        for (i = 1; i < n; i += 2) {
            if (a[i-1] > a[i]) Swap(&a[i-1], &a[i]);
        }
    } else {
        // Odd phase: compare odd-even pairs
        for (i = 1; i < n-1; i += 2) {
            if (a[i] > a[i+1]) Swap(&a[i], &a[i+1]);
        }
    }
}

```

```
}  
}
```

Parallel Version

```
c  
  
#pragma omp parallel for num_threads(thread_count) for  
(i = 0; i < n-1; i += 2) {  
    if (a[i] > a[i+1]) Swap(&a[i], &a[i+1]);  
}
```

BLOCK 9: PRODUCER-CONSUMER PATTERN

Concept

- **Producers:** Generate work/requests
- **Consumers:** Process work/requests
- **Queue:** Buffer between them

Message Passing with Queues

```
c  
  
// Sending a message  
mesg = random();  
dest = random() % thread_count; #pragma  
omp critical  
Enqueue(queue, dest, my_rank, mesg);  
  
// Receiving a message  
if (queue_size == 0) return;  
else if (queue_size == 1) {  
    #pragma omp critical  
    Dequeue(queue, &src, &mesg);  
} else {  
    Dequeue(queue, &src, &mesg);  
}
```

Termination Detection

```
c
```

```
queue_size = enqueued - dequeued;
if (queue_size == 0 && done_sending == thread_count) {
    return TRUE;
}
```

BLOCK 10: SYNCHRONIZATION

Barrier

```
c

#pragma omp barrier
// All threads must reach this before continuing
```

Critical Section

```
c

#pragma omp critical
{
    // Only one thread at a time
}
```

Atomic

```
c

#pragma omp atomic
x += 1; // Atomic operation
```

Locks

```
c

#include <omp.h>
omp_lock_t lock;

// Initialize
omp_init_lock(&lock);

// Use
omp_set_lock(&lock);
// Critical section
```

```
omp_unset_lock(&lock);

// Destroy
omp_destroy_lock(&lock);
```

Important Rules

1. Don't mix mutual exclusion methods for the same section
2. No guarantee of fairness
3. Beware of nesting locks (deadlock risk!)

BLOCK 11: MATRIX-VECTOR MULTIPLICATION

The Operation

text

```
y[i] = A[i][0]*x[0] + A[i][1]*x[1] + ... + A[i][n-1]*x[n-1]
```

Parallel Code

c

```
#pragma omp parallel for num_threads(thread_count) \ default(none)
private(i, j) shared(A, x, y, m, n)
for (i = 0; i < m; i++) {
    y[i] = 0.0;
    for (j = 0; j < n; j++) { y[i]
        += A[i][j] * x[j];
    }
}
```

Performance Results

Threads	8,000,000×8	8000×8000	8×8,
1	0.322s	0.264s	0.33
2	0.219s	0.189s	0.30
4	0.141s	0.119s	0.30

BLOCK 12: OPENMP SUMMARY

Key Directives

Directive	Purpose
<code>#pragma omp parallel</code>	Create thread team
<code>#pragma omp for</code>	Parallelize a for loop
<code>#pragma omp parallel for</code>	Both combined
<code>#pragma omp critical</code>	Mutual exclusion
<code>#pragma omp atomic</code>	Atomic operation
<code>#pragma omp barrier</code>	Synchronize threads
<code>#pragma omp single</code>	Execute by one thread

Key Clauses

Clause	Purpose
<code>num_threads(n)</code>	Set thread count
<code>reduction(op:var)</code>	Combine results
<code>private(list)</code>	Each thread has own copy
<code>shared(list)</code>	All threads access same
<code>default(none)</code>	Must specify all
<code>schedule(type,size)</code>	Loop distribution

Best Practices

1. Always check for data dependencies
2. Use reduction instead of critical when possible
3. Choose appropriate schedule type
4. Avoid nested locks
5. Compile with `-fopenmp` flag

QUICK REFERENCE CARD

Compile and Run

```
bash

gcc -fopenmp program.c -o program
./program thread_count
```

Basic Hello World

```
c

#include <omp.h>
#pragma omp parallel
{
    int rank = omp_get_thread_num();
    int total = omp_get_num_threads();
    printf("Thread %d of %d\n", rank, total);
}
```

Common Patterns

```
c

// Reduction
#pragma omp parallel for reduction(+:sum) for
(i = 0; i < n; i++) sum += f(i);

// Critical Section
#pragma omp critical
global_var += local_var;

// Custom Schedule
#pragma omp parallel for schedule(dynamic, 10)
for (i = 0; i < n; i++) work(i);

// Scope Control
#pragma omp parallel default(none) private(i) shared(n, data)
{
    // ...
}
```